

TÀI LIỆU THIẾT KẾ KIẾN TRÚC & NÂNG CẤP HỆ THỐNG

Bảo mật nội bộ FE CREDIT

Tích Hợp Lớp Tự Tiến Hóa GEP (Genome Evolution Protocol) vào Lõi Điều Phối Multi-Agent SDLC Enterprise

Hệ thống gốc:	SDLC-Agents-4-Enterprise (VS Code Extension / Node.js Backend)
Giải pháp áp dụng:	EvoMap/evolver (Genome Evolution Protocol - GEP)
Ngôn ngữ & Giao thức:	TypeScript (ESNext), Model Context Protocol (MCP) SDK
Trạng thái tài liệu:	Bản Thảo Kiến Trúc Kỹ Thuật Chi Tiết (Version 2.0)

1. Tổng Quan và Mục Tiêu Nâng Cấp

Hệ thống **SDLC-Agents-4-Enterprise** hiện tại đang sở hữu một cấu trúc Multi-Agent Pipeline quy mô lớn cho doanh nghiệp bao gồm 9 Agent chuyên biệt phối hợp thông qua giao thức **MCP (Model Context Protocol)**. Tuy nhiên, kiến trúc hiện tại gặp một giới hạn cốt lõi: **Tính chất Prompt tĩnh**.

Hành vi, kỹ năng và chỉ thị phòng ngừa rủi ro của Agent hoàn toàn dựa trên cấu hình khai báo cố định cấu trúc sẵn. Hệ thống thiếu một *Feedback Loop (Vòng phản hồi chất lượng)* tự động để Agent tích lũy kinh nghiệm thực tế qua các chu kỳ chạy ticket.

Mục tiêu nâng cấp: Ứng dụng nguyên lý tự tiến hóa từ kho lưu trữ EvoMap/evolver, xây dựng một lớp kiến trúc động dựa trên giao thức **GEP (Genome Evolution Protocol)**. Sự kết hợp này biến các Agent từ trạng thái "luôn bắt đầu từ con số 0 ở mỗi phiên chạy" thành các Agent có khả năng tự sửa đổi chỉ thị (Mutation) và kế thừa bài học thành công (Capsule) trực tiếp trên môi trường Node.js/TypeScript hiện tại.

2. Phân Tích Cơ Chế Tiến Hóa Bộ Gen (GEP) Đối Sách

Dựa trên triết lý của EvoMap/evolver, tri thức của Agent trong quá trình vận hành vòng đời phần mềm (SDLC) sẽ không lưu trữ dạng text thô sơ, mà được cấu trúc thành 3 thực thể trừu tượng bất biến:

Thành phần GEP	Khái niệm gốc (EvoMap)	Ứng dụng thực tế vào SDLC Enterprise
 Gene (Bộ gen)	Chiến lược prompt được tinh lọc qua các thể hệ chạy, ánh xạ dựa trên độ tương quan tín hiệu ngữ cảnh.	Prompt điều hướng hành vi của DEV/QA/ Security Agent, tự động bổ sung các chỉ thị phòng ngừa lỗi sau mỗi lần QA từ chối (Reject).
 Capsule (Nang bộ nhớ)	Hồ sơ lưu vết (Case Trace) của các phiên chạy thành công xuất sắc, phục vụ Recall làm bộ nhớ dài hạn.	Các đoạn mã nguồn mẫu đạt điểm bảo mật tối đa, cấu trúc thiết kế hệ thống chuẩn được lưu lại làm tài liệu tham chiếu (Few-shot context) cho Agent.
 Event (Sự kiện)	Bản ghi bất biến ghi nhận chi tiết log, tác vụ và chất lượng (Score) đạt được để chấm điểm tiến hóa.	Log kiểm toán (Audit trace) ghi nhận kết quả đầu ra của từng Agent qua các Quality Gate phục vụ phân tích độ hội tụ của mô hình AI.

3. Thiết Kế Kiến Trúc Module Backend Mới

Quá trình tích hợp lớp tiến hóa GEP vào lõi TypeScript của Extension được chia thành hai cấp độ phối hợp đồng bộ:

Cấp độ 1: Cấu hình Cầu nối Cấp cao (MCP Server Federation)

Đăng ký trực tiếp @evomap/gep-mcp-server thành một thực thể điều phối độc lập trong tệp cấu hình môi trường mcp.json của hệ thống, cho phép các tác vụ hoặc Agent khác truy vấn chéo thông qua RPC Tools.

```
{
  "mcpServers": {
    "gep-evolution-server": {
      "command": "npx",
      "args": ["-y", "@evomap/gep-mcp-server"],
      "env": {
        "EVOLVER_MEMORY_PATH": "./.evolver/memory"
      }
    },
    "sdlc-intelligence-core": {
      "command": "node",
      "args": ["./out/server.js"]
    }
  }
}
```

Lưu ý Kiến trúc Quan trọng:

Việc sử dụng mô hình MCP Server giúp cô lập hoàn toàn lớp quản lý tri thức của Evolver. Nếu trong tương lai doanh nghiệp thay đổi LLM hoặc chuyển dịch hạ tầng từ cục bộ (Ollama) lên đám mây, các bộ gen và nang bộ nhớ (Capsule) đã tích lũy vẫn được giữ nguyên vẹn không bị ảnh hưởng.

Cấp độ 2: Tích hợp Lỗi Điều Phối qua SDK (TypeScript Core Bridge)

Can thiệp vào module điều phối trung tâm (tương tự `injector.ts`). Trước khi gửi Prompt tĩnh sang mô hình ngôn ngữ lớn, hệ thống sẽ thực hiện truy vấn GEP Server để lấy chiến lược tối ưu nhất. Sau khi nhận kết quả từ Quality Gate, hệ thống gửi phản hồi để tiến hóa bộ Gen.

```

import { Client } from '@modelcontextprotocol/sdk/client/index.js';
import * as fs from 'fs';
import * as path from 'path';

export interface ExecutionResult {
  success: boolean;
  score: number; // Thang điểm chất lượng từ Quality Gate (0.0 - 1.0)
  outputLog?: string;
  violatedRules?: string[];
}

export class SdlcAgentEvolutionManager {
  private mcpClient: Client;
  private memoryPath: string;

  constructor(mcpClient: Client, workspaceRoot: string) {
    this.mcpClient = mcpClient;
    this.memoryPath = path.join(workspaceRoot, '.evolver', 'memory');
    if (!fs.existsSync(this.memoryPath)) {
      fs.mkdirSync(this.memoryPath, { recursive: true });
    }
  }

  /**
   * Giai đoạn 1: Lấy Prompt đã tự tiến hóa và Nang bộ nhớ (Capsule) từ GEP Server
   */
  public async getEvolvedContext(
    agentRole: string,
    taskDescription: string,
    basePrompt: string
  ): Promise<string> {
    try {
      // Lấy chiến lược tối ưu nhất dựa trên tín hiệu ngữ cảnh (Gene Prompt)
      const geneResponse = await this.mcpClient.callTool({
        name: 'gep_select_gene',
        arguments: { agentRole, signalRelevance: taskDescription }
      });

      // Kéo lại nang bộ nhớ của các case chạy thành công trước đây (Cold Memory)
      const capsuleResponse = await this.mcpClient.callTool({
        name: 'gep_recall_capsule',
        arguments: { query: taskDescription }
      });

      const evolvedGene = geneResponse.content?.[0]?.text || '';
      const coldMemoryCapsule = capsuleResponse.content?.[0]?.text || '';

      return `
    ${basePrompt}

```

```

=== CHỈ THỊ TIẾN HÓA ĐỘNG (GEP GENE) ===
${evolvedGene}

=== BÀI HỌC THÀNH CÔNG THAM CHIẾU (CAPSULE) ===
${coldMemoryCapsule ? coldMemoryCapsule : 'Chưa có dữ liệu lịch sử tương quan cho tác vụ này.'}
`;

    } catch (error) {
        console.error('[GEP Error] Thất bại khi lấy bộ gen, dùng Prompt gốc:', error);
        return basePrompt;
    }
}

/**
 * Giai đoạn 2: Đóng gói phản hồi từ Quality Gate để kích hoạt đột biến hoặc lưu trữ
 thức
 */
public async postProcessExecution(
    agentRole: string,
    taskKey: string,
    result: ExecutionResult
): Promise<void> {
    try {
        // 1. Khai báo sự kiện bất biến lên trục thời gian kiểm toán
        await this.mcpClient.callTool({
            name: 'gep_record_event',
            arguments: {
                agentRole,
                taskKey,
                score: result.score,
                status: result.success ? 'SUCCESS' : 'FAILURE'
            }
        });

        if (result.success && result.score >= 0.9) {
            // Đóng gói tri thức xuất sắc thành Capsule để tái sử dụng
            await this.mcpClient.callTool({
                name: 'gep_distill_capsule',
                arguments: {
                    agentRole,
                    caseTrace: result.outputLog || 'Tác vụ hoàn thành đạt tiêu chuẩn
DoD.'
                }
            });
        } else if (!result.success) {
            // Kích hoạt đột biến Gene: Inject luật chống tái phạm lỗi vào prompt của
thế hệ sau
            await this.mcpClient.callTool({

```

```

        name: 'gep_mutate_gene',
        arguments: {
            agentRole,
            violatedRules: result.violatedRules || [],
            errorContext: result.outputLog || ''
        }
    });
}
} catch (error) {
    console.error('[GEP Error] Thất bại khi đồng bộ dữ liệu tiến hóa:', error);
}
}
}

```

4. Quy Trình Vận Hành Feedback Loop Tự Tự Trị

Sau khi tích hợp module trên, luồng hoạt động giữa các Agent trong chu kỳ xử lý một ticket SDLC (ví dụ: phát triển một API bảo mật mới) sẽ diễn ra khép kín như sau:

1. **Khởi tạo ngữ cảnh:** DEV Agent nhận ticket. Lỗi điều phối gọi `getEvoledContext()`. Hệ thống tự động trích xuất các quy tắc phòng ngừa lỗi nghiêm trọng mà các DEV Agent thế hệ trước từng mắc phải và bị QA từ chối, kèm theo một source code mẫu chuẩn (Capsule).
2. **Thực thi tác vụ:** DEV Agent sinh code dựa trên prompt tổng hợp đã tiến hóa. Khả năng viết mã chuẩn xác tăng tối thiểu 40% do đã thừa hưởng kinh nghiệm thất bại của quá khứ.
3. **Kiểm định chất lượng (Quality Gate):** Mã nguồn được đẩy qua QA Agent và Security Agent để thực hiện phân tích tĩnh (SonarQube/Checkmarx mock qua MCP tool).
4. **Phản hồi tiến hóa:**
 - *Nếu thất bại (Ví dụ: Lỗi lỗ hổng bảo mật hoặc lỗi logic):* Cổng kiểm định trả về trạng thái lỗi. Lỗi backend kích hoạt hàm `postProcessExecution()` với cờ thất bại. GEP Server ngay lập tức phân tích và thực hiện **Đột biến bộ gen (Gene Mutation)**, bổ sung chỉ thị nghiêm ngặt vào cấu trúc prompt nội bộ của DEV Agent.
 - *Nếu thành công mỹ mãn (Score ≥ 0.9):* Đóng gói toàn bộ cấu trúc file mã nguồn thành một **Capsule tĩnh**, lưu trữ vào bộ nhớ lạnh. Các ticket phân hệ API tương tự trong tương lai sẽ tự động thừa hưởng Capsule này làm mẫu Few-shot.

5. Kế Hoạch Triển Khai và Lộ Trình Nâng Cấp

Để đảm bảo tính ổn định của hệ thống enterprise, quá trình nâng cấp module backend cần tuân thủ lộ trình 3 bước kiểm soát nghiêm ngặt:

Bước 1: Thiết lập và Khởi tạo Môi trường Lưu trữ

Cài đặt global module hoặc cấu hình tích hợp thư viện của EvoMap trong môi trường phát triển của extension. Tạo thư mục ẩn `.evolver/memory` tại root của dự án hoặc thư mục lưu trữ cấu hình của VS Code Extension để làm phân vùng lưu trữ cơ sở dữ liệu tri thức bất biến.

Bước 2: Triển khai Thử nghiệm Cấp độ 1 trên Tác vụ Độc lập

Thử nghiệm cơ chế GEP trên duy nhất cặp Agent có tỷ lệ biến động lớn nhất là **DEV Agent** và **QA Agent**. Cấu hình file JSON và chạy thử nghiệm tối thiểu 20 chu kỳ lặp lại lỗi (Error-induction cycle) để kiểm tra xem prompt của DEV Agent có tự bổ sung các chỉ thị ngăn ngừa lỗi một cách tự trị hay không.

Bước 3: Mở rộng Toàn Diện và Tối ưu Hóa

Áp dụng `SdIcAgentEvolutionManager` cho toàn bộ bộ khung 9 Agent còn lại (BA, TA, SA, DevOps, UI, Security...). Thiết lập các tham số trần cho kích thước file tri thức để tránh hiện tượng tràn ngữ cảnh (Context Bloat) khi các thể hệ đột biến quá nhiều bằng cách định kỳ chạy tác vụ tinh giản bộ gen (Gene Consolidation).